Tactile Sensor - Instruction Manual

August 2019

1 General Limitations for using the Sensors

- Applying too high forces or pressures will destroy the sensor module and will void the warranty. Keep the forces (applied to the whole surface of the Sensor Module) and pressures (depending on your contact area) below those values:

  z-axis (forces applied normal to the sensor's surface) limits: 10N/25kPa

  x-axis and y-axis: the sum of the shear forces has to stay below 6N/15kPa

  For example, if you contact only half of the Sensor Module's surface, only apply 5N normal force.

  Furthermore, apply forces only to the sensor surface, not to the sides of the sensor module.

- As a skin sensor, the contact geometry always influences the measurements. Therefore, we do not calibrate the sensors, as any calibration would be valid only for the same contact shape. We suggest that you use machine learning techniques or your algorithms to get various information out of the sensor measurements, relevant for your application.

- As our sensor uses magnetic field changes induced by the skin deformation as its sensing principle, other magnetic fields (including the earth magnetic field), nearby magnets, or nearby ferromagnetic materials can influence the sensor measurements. Also crosstalk between two of our sensor modules is possible. Please confirm within the inspection period (1 month from receipt of the product) with your application if those influences are prohibitory for your application. To counteract those influences, please also consider that a reference sensor could be used.

- Never bend the sensor modules. When you install the sensor modules on your robot, glue them to a sturdy and flat surface with thin double-sided sticky tape. Make sure to provide flat support to the whole backside of the sensor module.

2 Requirements

For using the sensors:

- Operating System: tested & working on Windows 7 & 10.
- Programming Language: Depending on your the compatibility of your CAN-USB device. We have tested our sensor with CAN-USB/2 from ESD and PCAN-USB from PEAK System.
  - ESD provides API for .NET, C#, Python, VC, Visual Basic, BC, LabVIEW, Linux, etc. Please refer to ESD website for more information.
  - Peak System provides API for C#, Python, C, Visual Basic, Linux, etc. Please refer to PEAK System website for more information.

    Currently, we recommend ESD to achieve a higher measurement frequency.
- PC with USB 2.0 port.

For the provided visualization software:

- OS: Windows 7 SP1 / 8 / 8.1 / 10

- .NET 2.0 or higher
- ESD CAN driver for Windows 2.5.x or later

3 Hardware Introduction

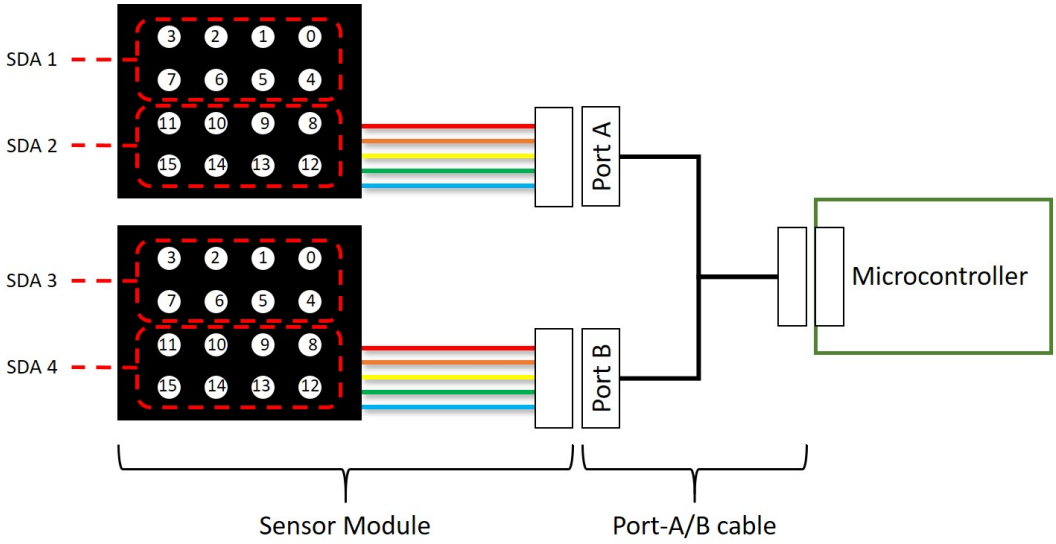

Figure 1: The connection between Sensor Module, Port-A/B cable, the microncontroller, and their respective SDA numbers and taxel numbers.

3.1 Sensor Module

Each Sensor Module has 16 sensing points (taxels) in total. Each taxel measures 3-axis skin deformation. The sampling rate is 250 Hz. Each measurement has 16-bit (8 Most Significant Bit/ MSB and 8 Least Significant Bit/ LSB) resolution per axis. Please see Figure.1 for the taxels' number and their position.

3.2 Port-A/B cable

The Port-A/B cable is used for connecting 2 Sensor Modules to 1 microcontroller. A label at the Port-A/B's end identify which Port a Sensor Module is connected to. Depending on this, the SDA number changes.

When a Port-A/B cable is not used to connect a Sensor Module to a microcontroller, it will be treated as the Sensor Module is connected via Port-A. In other words, the SDA number of the Sensor Module will be SDA1 and SDA2.

3.3 Microcontroller

The pre-programmed microcontroller is used to start the communication, configuring, and collecting the data of the Sensor Module. The microcontroller can be connected to the Sensor Module through its 8-pin port. In the current version of the Sensor Module, 2 of the module can be connected to one microcontroller via the provided Port-A/B cable.

On the microcontroller, there are two 4-pin ports (VDD, D+, D-, GND). One of those ports is for the communication between the microcontroller and the CAN/USB converter. The other one is for a daisy-chain communication between microcontrollers through a CAN protocol. These ports are interchangeable.

Up to 4 microcontrollers with Sensor Modules can be daisy-chained. Note that when more than 4 Sensor Modules and 2 microcontrollers are connected to 1 CAN bus, an additional 5VDC power source is needed to be connected to 1 of the CAN port of the microcontroller on the furthest end.

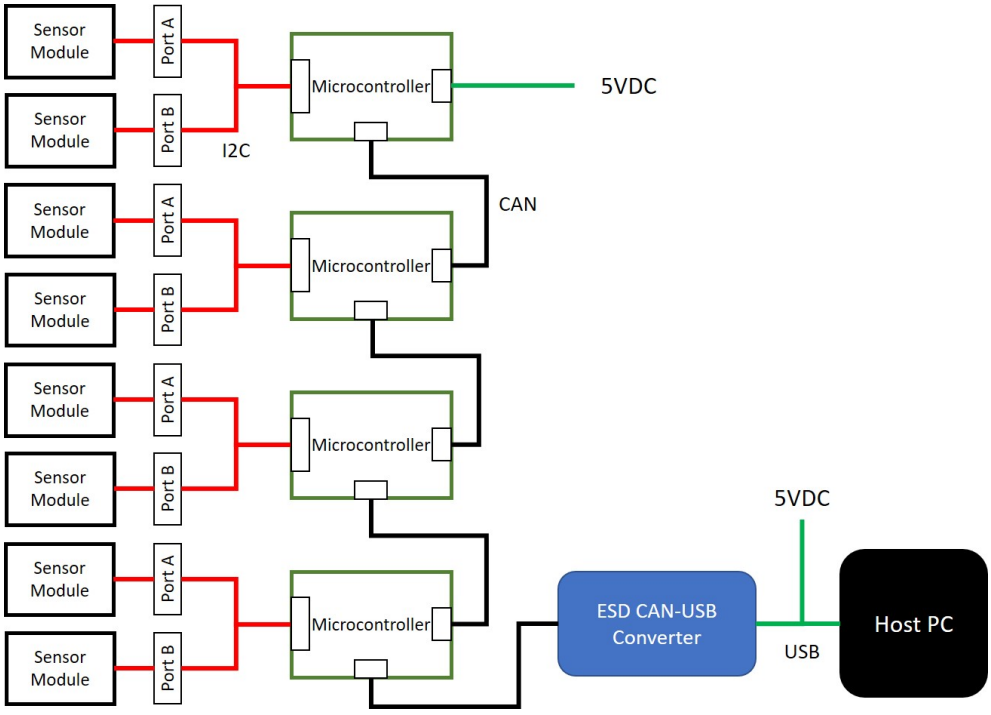

Figure 2: Connection diagram.

3.4 ESD CAN/USB Interface

This device interfaces a PC with the microcontroller. It is connected to the PC with a serial bus (USB). This device was developed by ESD and can be purchased separately from https://esd.eu/en/products/can-usb2. The driver is also available from the prescribed link.

3.5 CAN to DE-9 cable

This cable connects the microcontroller (4-pin connector) to the ESD CAN/USB converter (DE-9 connector). The 4 pin wires are for transmitting data to the ESD CAN/USB interface. The USB cable is only for power (5V).

4 Setup & Installation

4.1 Connecting the hardware

1. Plug the 8-pin wires of one Sensor Module's into a microcontroller. The Port-A/B cable can be used to connect two Sensor Modules to one microcontroller.
2. Connect the 4-pin connector of the CAN/DE-9 cable to 1 of the 2 4-pin port of the microcontroller.
3. Connect the DE-9 connector of the CAN/DE-9 cable to the CAN/USB Interface.
4. Plug the USB cable of the CAN/USB Interface into any of the USB ports of your PC.
5. Plug the USB power cable of the CAN/DE-9 cable into a PC or USB wall adapter (5V). The power indicator (red LED) of the microcontroller should flash.
6. (Optional) Up to 8 Sensor Modules can be daisy-chained as in Fig. 2.

4.2 Driver Installation

4.2.1 CAN SDK

Download the necessary files and run CAN_SDK.exe from ...\CAN USB driver\CAN_SDK and follow the instructions. This will install the ESD CAN/USB libraries and sample programs required for the next step.

4.2.2 ESD CAN/USB driver

After plugging in the USB cable, open the device manager from the control panel. In the USB section, make sure that the device is detected as an unknown device. Right click the unknown device and specify the driver location as ...\CAN USB Driver. If it is successful, the unknown device should turn into "CAN Interface - CAN USB/2".

4.2.3 NTCAN.NET

The NTCAN.NET (NtcanNets.dll) can be downloaded from this link (https://esd.eu/en/products/ntcannet). This is used in our sensor visualization sample software. Otherwise, the NtcanNets.dll can be found in the provided software.

5 Explanation of CAN ID and CAN message

Here we use CANreal application provided by ESD to make the explanation easy to understand. Run the CANreal application. The application is installed as part of SDK and can be found by default at C:\Program Files\ESD\CAN\SDK\bin32\CANreal. Configure it as in Figure.3.

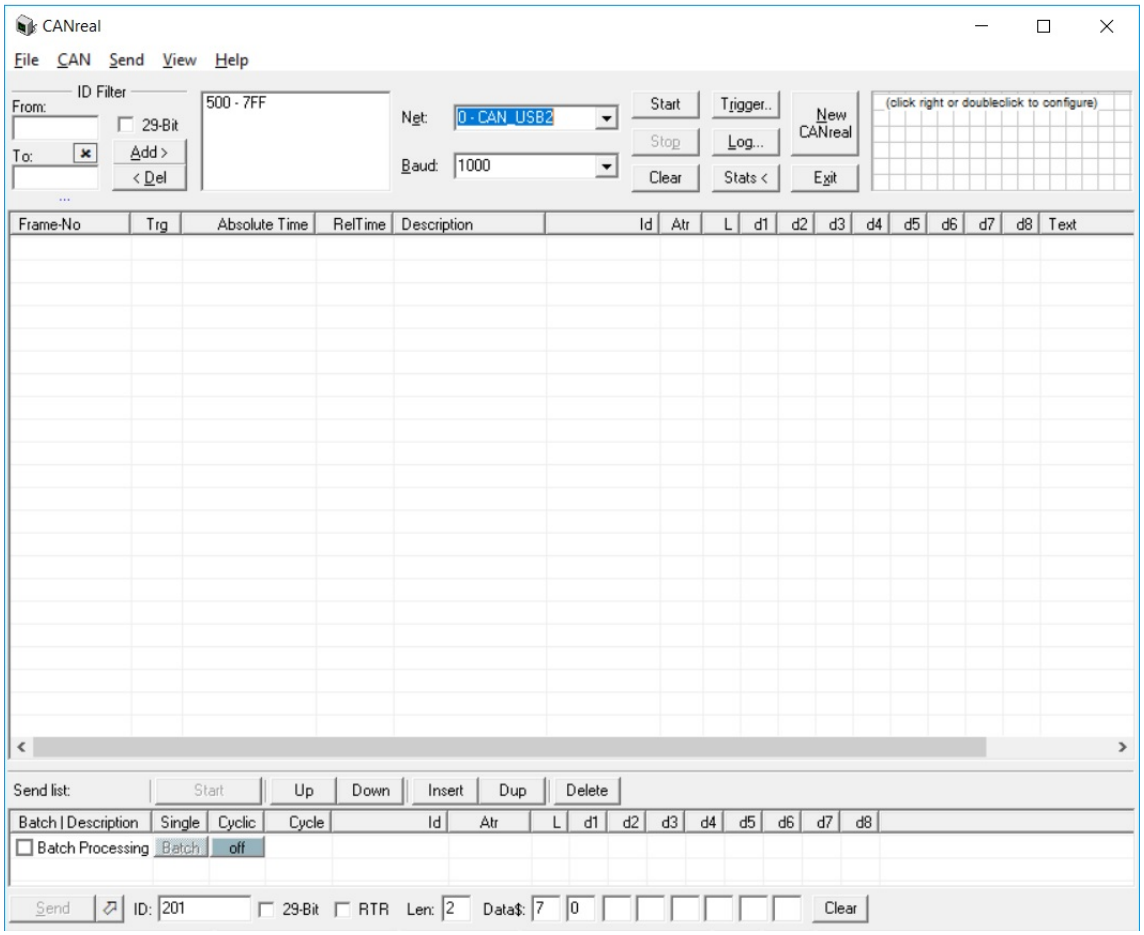

Figure 3: CANreal configuration.

Select a detected CAN/USB device by choosing it from the "Net" drop-down menu. If there is nothing that can be selected, the device may not be plugged or the driver is not installed properly. Configure the "Baud" to 1000 then click "Start".

5.1 Outgoing CAN ID

Please enter the ID in the lower left corner of the CANreal with 20x (hexadecimal). x is the ID number written on the back of the microcontroller. For example, if the ID is 1, the CAN ID should be 201.

5.2 Outgoing CAN message (Trigger the microcontroller)

To trigger the microcontroller, 2 bits of data must be sent. The first bit is a command header (pre-defined in the microcontroller). The value is 7. The second bit is a Boolean value, 0 means start sending measurement data, 1 means stop. Therefore, to instruct the microcontroller to start sending the sensor's measurement, "Len" and "Data$ 1 and 2" should be configured as 2, 7, and 0 respectively. To instruct the microcontroller to stop sending the sensor's measurement, "Data$ 2" need to be 1. Example of this triggering step is shown in Table 1.

In the case where several microcontrollers are daisy-chained on the same CAN bus, you need to iteratively trigger start and stop the microcontrollers before the data from the sensor connecting to each microcontroller can be read.

Microcontroller ID	CAN ID	Len	Data$	
			1	2
1	201	2	7	0 = start, 1 = stop
2	202	2	7	0 = start, 1 = stop

Table 1: Example of CAN ID and message for triggering the microcontroller

Bit	11	10	9	8	7	6	5	4	3	2	1	0
Group	Microcontroller ID				SDA number - 1				Taxel Number			

Table 2: Incoming CAN ID Structure

After properly configuring the CANreal, click send. The blue LED of the microcontroller should start blinking. You can also see the incoming data in CANreal. If there is no response at all, the cables may not be properly connected or there is a problem with the sensor.

5.3 Incoming CAN ID structure (slave-to-master)

Each incoming CAN message comes with its ID. The ID represents the number of microcontroller and the taxel of the Sensor Module connecting to that microcontroller. The meaning of the ID is as shown in Table 2.

- "Microcontroller ID" is pre-defined. The number can be found on the back of each microcontroller.
- "SDA number - 1" is defined from whether Port A or Port B of the Port-A/B cable that the Sensor Module is connected to. If a Sensor Module is connected to Port A, the SDA number will be 1 and 2 depending on the taxel. See Figure 1 for more detail.
- "Taxel Number" is defined as per below in hexadecimal.
  - Taxel 0 - 3 : 0x0 - 0x3
  - Taxel 4 - 7 : 0x8 - 0xB
  - Taxel 8 - 11 : 0x0 - 0x3
  - Taxel 12 - 15: 0x8 - 0xB

Therefore, the incoming CAN IDs of the taxels on the Sensor Module which is connected to the microcontroller **ID1** via **Port A**, are as follow.

- Taxel 0 - 3 : 0x100 - 0x103
- Taxel 4 - 7 : 0x108 - 0x10B
- Taxel 8 - 11: 0x110 - 0x113
- Taxel 12 - 15: 0x118 - 0x11B

For the Sensor Module that is connected via **Port B** of the same microcontroller **ID1**, the incoming CAN IDs of the taxels are as follow.

- Taxel 0 - 3 : 0x120 - 0x123
- Taxel 4 - 7 : 0x128 - 0x12B
- Taxel 8 - 11: 0x130 - 0x133
- Taxel 12 - 15: 0x138 - 0x13B

Note that if a Sensor Module is connected directly to a microcontroller without any Port-A/B cable, it will be treated as connecting to Port A.

5.4 Incomming CAN Message structure

Each incoming CAN message contain 8-byte data. The data compose of 3-axis components of contact measurement. The structure of the data is as follows.

- 1st byte : Not used
- 2nd byte : X-axis MSB
- 3rd byte : X-axis LSB
- 4th byte : Y-axis MSB
- 5th byte : Y-axis LSB
- 6th byte : Z-axis MSB
- 7th byte : Z-axis LSB
- 8th byte : SDA number

By combining the MSB and LSB part of each axis, the 16-bit measurement can be acquired.

6 Running the Sample Code

This sample code demonstrates how the sensor data can be interfaced to a different programming environment. In this sample, we use a visualization software written in C# and WPF with .NET framework.The visualization software works with ESD CAN/USB only. This software only works with the Microcontroller ID 1. Any Sensor Module can be connected to either port A or B of this microcontroller for testing the sensor modules.

1. Make sure that the Sensor Module is powered and the ESD CAN/USB is connected to a PC. If you have more than one Sensor Module, please connect all the sensors with all the microcontrollers, and connect all the microcontrollers together.
2. Wait 5 seconds to allow the microcontroller to finish initialization. This can be seen from the solid blue LED. If the microcontroller is still initializing, the blue LED will blink.
3. Double-click "XelaSkinVisualization.exe" in XelaSkinVisualization\bin\Debug to run the visualization software.
4. You can click the "Reactivate uC" button to reactivate the microcontroller in the case that there is no data streaming. Another indication is that the LED of the microcontroller is not blinking.
5. You can click the "Recal Baseline" button to recalculate the sensor's baseline. Please don't touch any Sensor Module and wait for a few seconds for this to complete.
6. Afterward, the dots representing the contact will appear when their taxels are touched. The diameter of the circle represents normal strength (z-axis). The movements of the circle in x and y-axis directions represents shear strength.

Users can write another visualization software or a program to read the measurement from the sensor in other languages and platforms such as Python.